# FIXPROOF

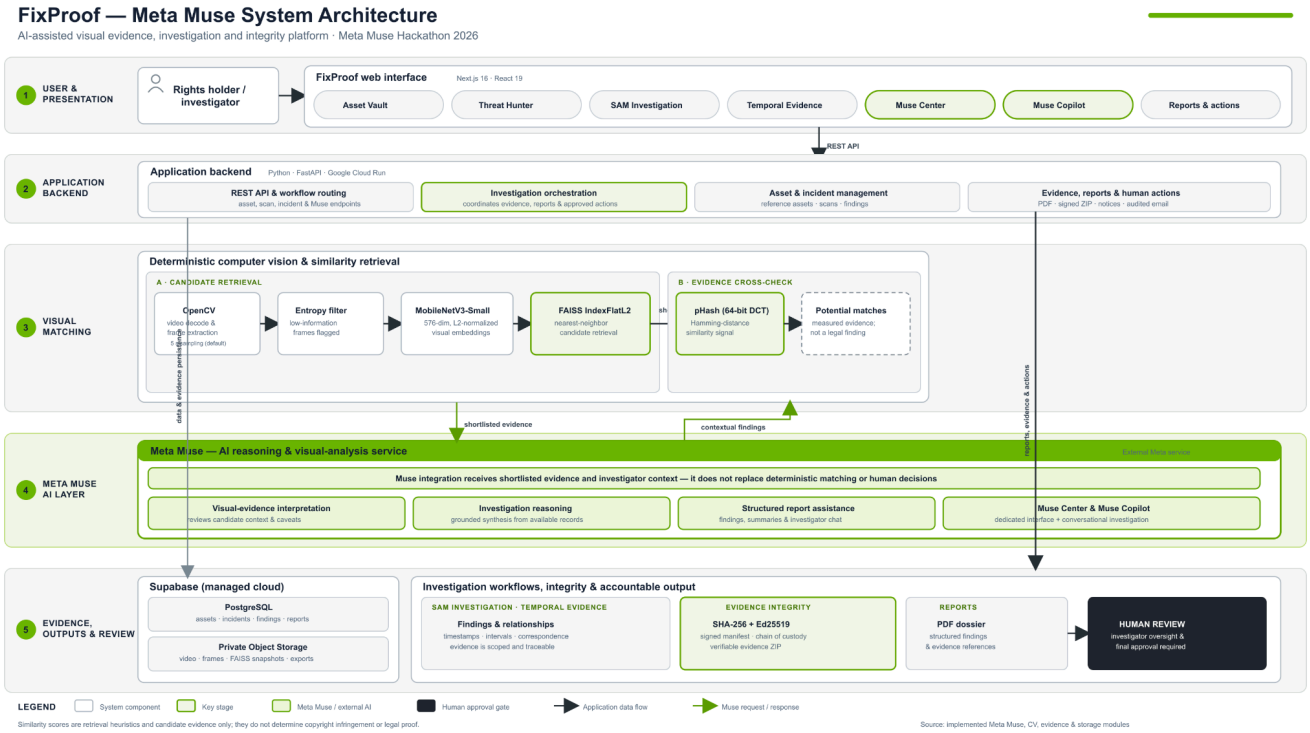
## AI Model Usage & Meta Muse Integration

*Technical documentation for the FixProof sports-media visual verification platform*

| Resource / Field | Link / Information |
| --- | --- |
| Repository | https://github.com/varunchauhan12/fixproof-muse-hackathon |
| Live frontend | https://fixproof-frontend-1097663016023.europe-west1.run.app |
| Live backend | https://fixproof-muse-hackathon-git-1097663016023.europe-west1.run.app |

# 1. Purpose and Scope

FixProof combines computer-vision candidate retrieval with AI-assisted investigation. The visual matching pipeline identifies potential correspondences between reference and suspect media; Meta Muse models provide additional analysis in the workflows that invoke them. Model output supports human review and does not independently determine whether copyright infringement occurred.




# 2. Integration Overview

The backend accesses Meta Muse models through the Meta Model API using an API-compatible client. The integration should be described as separate model calls coordinated by the FixProof backend—not as models autonomously communicating with one another.

Provider endpoint reported in the project materials: https://api.meta.ai/v1/

Confirm endpoint and model configuration against the current deployment before publishing. Never include API keys, authorization headers, or secret environment-variable values in this document.

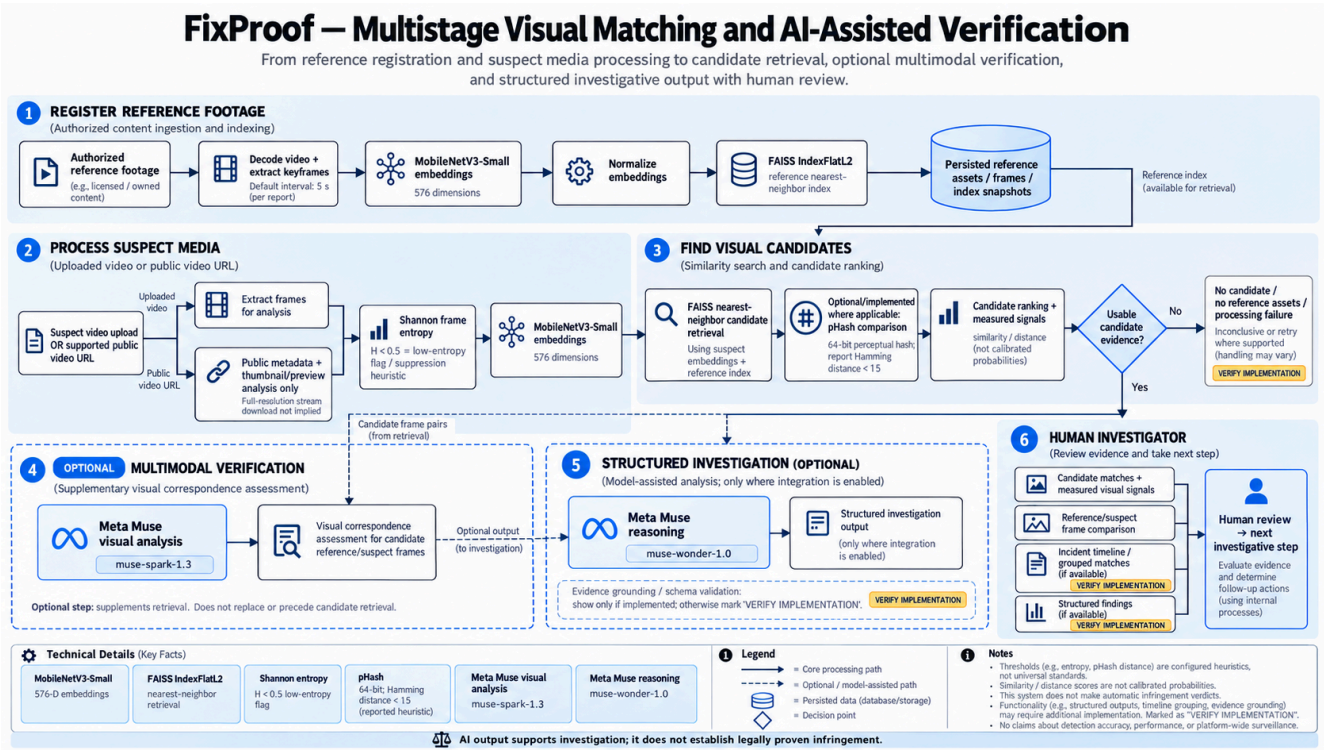
# 3. Model Roles and Source Call Sites

| Model ID | Reported role | Source call sites reported by repository audit |
|---|---|---|
| muse-wonder-1.0 | Structured investigation analysis and document drafting. | backend/app/providers/meta.py; investigation and document workflows |
| muse-spark-1.3 | Pairwise visual assessment of shortlisted reference and suspect frames. | backend/app/providers/meta.py; visual verification workflow |
| muse-wonder-1.0 | Detailed report synthesis using the configured Muse reasoning model. | backend/app/detailed_report.py — generate_detailed_report |

Verification note: these model roles and call sites were reported by a repository inspection. Before publication, confirm the current source still matches this table and distinguish code presence from a live inference test.

## 4. Where AI Fits in the Pipeline

1. Reference and suspect media are processed into frames and visual representations.
2. MobileNetV3-Small generates 576-dimensional embeddings for the vector-retrieval workflow.
3. FAISS IndexFlatL2 retrieves candidate frames; entropy filtering and perceptual hashing provide additional heuristic signals.
4. Where enabled, Muse vision (muse-spark-1.3 by default) assesses shortlisted frame pairs.
5. Muse reasoning (muse-wonder-1.0 by default) can produce structured investigation output for relevant workflows.
6. The detailed-report model can synthesize a longer report where that integration is enabled.
7. An investigator reviews candidate evidence and any generated findings before taking further action.




## 5. Structured Output and Grounding

The supplied project report describes structured JSON output and validation of evidence references. Only retain detailed claims about schema enforcement, allowed evidence IDs, or rejection behavior after checking the relevant validator and tests in the current source. Historical detailed report example (generated under the prior model configuration).

**https://drive.google.com/file/d/1oGwy5tubQ2AErlyD10gLbS_lamhBNKwO/view?usp=sharing**

## 6. Detailed Reports

The detailed-report workflow in backend/app/detailed_report.py uses the configured Muse reasoning provider. The default is muse-wonder-1.0; configuration can override it. If AI synthesis fails, deterministic output may be retained with failure metadata rather than presented as successful AI output.

PDF generation is handled with ReportLab according to the project materials. A generated report should distinguish observed media signals, model-generated interpretation, and investigator conclusions.

## 7. Human Oversight and Responsible Use

- Candidate similarity is not proof of copyright infringement.

- Similarity distances and thresholds are heuristics unless independently calibrated and evaluated.
- AI output can be incomplete or incorrect and requires human review.
- Cryptographic signatures can help verify covered data integrity; they do not establish that an allegation is true or that evidence is legally admissible.
- External notices or licensing communications should only be sent after the required human confirmation in the implemented workflow.

## 8. Configuration

| Variable name | Purpose | Publication note |
| --- | --- | --- |
| MODEL_API_KEY | Authenticates requests to the Meta Model API. | Never publish the value. |
| MUSE_REASONING_MODEL | Optional override for the Muse reasoning model. | Verify default in current source. |
| MUSE_VISION_MODEL | Optional override for the Muse vision model. | Verify the configured vision model. |

Other configuration variables may be required for database, object storage, signing, and email features. Refer to the repository's actual environment example for the complete list; do not add secret values to this document.

## 9. Validation Evidence

A model ID in source code demonstrates an intended integration path, not that a live inference request succeeded. Record live model tests only when the request was actually executed and its result reviewed.

| Validation item | Evidence to record | Status |
| --- | --- | --- |
| Model call site | Source file and function | Inspect source |
| Successful inference | Date, model ID, sanitized success result | Add only if tested |
| Failure handling | Observed error/fallback test or relevant unit test | Add only if verified |
| End-to-end workflow | Real candidate frames through AI output and human review | Add only if tested |

## 10. Limitations

The model-assisted stages operate on candidates produced by the visual retrieval workflow. URL screening is limited to content accessible to the implemented preview/metadata workflow and does not imply DRM bypass or unrestricted full-stream downloading. Audio fingerprinting is not described as an implemented capability in the supplied audit.

## 11. Assets to Add Before Submission

| Asset | Placement | Priority | Instructions |
|---|---|---|---|
| FixProof logo | Cover page | Optional | Insert a transparent PNG or SVG if available. |
| System architecture diagram | Section 1 or 2 | Optional here; recommended in main project report | Use fixproof-architecture.png. |
| Visual verification / detection pipeline | Section 4 | Recommended | Use FixProof Visual Verification Workflow.png. |
| Sanitized structured report screenshot | Section 5 | Optional | Use a real application screenshot; do not fabricate output. |

## 12. Final Review Checklist

☐ Confirm model IDs and call sites against current source code.

☐ Verify model defaults and fallback behavior before describing them as facts.

☐ Do not publish secrets, private keys, or authorization headers.

☐ Insert only real diagrams and screenshots; remove unused placeholders.

☐ Distinguish source inspection, unit tests, live inference, and end-to-end validation.

☐ Avoid claims of legal adjudication, guaranteed accuracy, or legal admissibility.